

Глава 3

Новые средства языка

Ядро языка C++ и его библиотеку обычно стандартизировали параллельно. Благодаря этому библиотека могла использовать преимущества, предоставляемые новыми средствами языка, а язык мог улучшаться за счет опыта реализации библиотеки. По этой причине стандартная библиотека языка C++ всегда использует конкретные средства языка, которых могло не быть в предыдущих версиях стандарта.

Таким образом, язык C++11 отличается от языка C++98/C++03, а тот, в свою очередь, отличается от языка C++, существовавшего до начала стандартизации. Если не знать об этой эволюции, то можно удивиться новым языковым средствам, используемым в библиотеке. В настоящей главе дан краткий обзор новых средств языка C++11, играющих важную роль в проектировании, понимании и применении стандартной библиотеки C++11. В конце главы описываются некоторые средства, существовавшие до появления стандарта C++11, но не получившие широкой известности.

Когда я писал эту книгу (2010-2011 гг.), не все компиляторы поддерживали новые языковые средства стандарта C++11. В ближайшее время это положение, по-видимому, изменится, поскольку все основные поставщики компиляторов языка C++ принимали участие в процессе его стандартизации¹. Но некоторое время пользователи будут ограничены в выборе языковых средств, используемых в библиотеке. На протяжении книги все типичные и важные ограничения будут указываться в сносках.

3.1. Новые языковые средства стандарта C++11

3.1.1. Небольшие, но важные синтаксические уточнения

Во-первых, следует упомянуть о двух новых средствах языка C++11, которые являются небольшими, но важными для повседневного программирования.

Пробелы в выражениях с шаблонами

Требование помещать пробел между двумя закрывающими угловыми скобками в шаблонных выражениях отменено:

```
vector<list<int> >; // OK в каждой версии C++  
vector<list<int>>; // OK, начиная с версии C++11
```

В книге (и реальных программах) можно встретить обе эти формы.

¹ С текущим состоянием дел можно ознакомиться, например, по адресу <http://wiki.apache.org/stdcxx/C++0xCompilerSupport>. — *Примеч. ред.*

Ключевое слово `nullptr` и тип `std::nullptr_t`

Стандарт C++11 позволяет использовать ключевое слово `nullptr` вместо 0 или `NULL`, чтобы отметить тот факт, что указатель не ссылается ни на один объект (в отличие от ситуации, когда указатель не имеет определенного значения). Это новое средство языка позволяет избежать ошибок, возникающих, когда нулевой указатель интерпретируется как целочисленное значение. Например:

```
void f(int);

void f(void*);

f(0);           // вызов f(int)

f(NULL);        // вызов f(int), если NULL == 0,
                // и неоднозначность в противном случае

f(nullptr);     // вызов f(void*)
```

Слово `nullptr` — это новое ключевое слово. Константа, заданная с помощью ключевого слова `nullptr`, автоматически конвертируется в любой тип указателя, но не в целочисленный тип. Она имеет тип `std::nullptr_t`, определенный в заголовке `<cstddef>` (см. раздел 5.8.1), так что теперь есть возможность даже перегружать операторы для ситуаций, когда им передается нулевой указатель. Отметим, что тип `std::nullptr_t` относится к элементарным типам данных (см. раздел 5.4.2).

3.1.2. Автоматическое выведение типа с помощью ключевого слова `auto`

В языке C++11 можно объявить переменную или объект без указания их конкретного типа, используя ключевое слово `auto`.² Рассмотрим пример:

```
auto i = 42; // Переменная i имеет тип int
double f();
auto d = f(); // Переменная d имеет тип double
```

Тип переменной, объявленной с помощью ключевого слова `auto`, выводится из ее инициализатора. Например, в следующем коде требуется инициализатор:

```
auto i; // ОШИБКА: невозможно вывести тип переменной i
```

Разрешается использование дополнительных квалификаторов. Например:

```
static auto vat = 0.19;
```

Использование ключевого слова `auto` особенно полезно в ситуациях, когда тип является очень длинным, а выражение сложным. Например:

² Отметим, что `auto` — старое ключевое слово, унаследованное от языка C. В качестве противоположности ключевому слову `static` оно означало, что переменная является локальной, но никогда не использовалось, потому что по умолчанию все, что не объявлено как `static`, неявно считалось объявленным как `auto`.

```
vector<string> v;
...
auto pos = v.begin(); // переменная pos имеет тип
                      // vector<string>::iterator
auto l = [] (int x) -> bool { // Переменная l имеет тип лямбда-функции,
    ...,                    // принимающей int и возвращающей bool
};
```

Переменная `l` является объектом, который представляет собой лямбда-функцию (лямбда-функции рассматриваются в разделе 3.1.10).

3.1.3. Универсальная инициализация и списки инициализации

До появления стандарта C++11 программисты, особенно начинающие, легко могли запутаться в вопросах инициализации переменной или объекта. Инициализация могла осуществляться с помощью круглых или фигурных скобок, а также операторов присваивания.

По этой причине в стандарт C++11 включена концепция универсальной инициализации, означающая, что любая инициализация осуществляется с помощью единообразного синтаксиса. Эта конструкция использует фигурные скобки. Например:

```
int values[] { 1, 2, 3 };
std::vector<int> v { 2, 3, 5, 7, 11, 13, 17 };
std::vector<std::string> cities {
    "Berlin", "New York", "London", "Braunschweig", "Cairo", "Cologne"
};
std::complex<double> c{4.0,3.0}; // эквивалентно to c(4.0,3.0)
```

Список инициализации осуществляет так называемую *инициализацию значениями* (*value initialization*), подразумевающую, что каждая, даже локальная переменная элементарного типа, которая обычно имеет неопределенное начальное значение, инициализируется нулем (или константой `nullptr`, если переменная является указателем):

```
int i;      // Переменная i имеет неопределенное значение
int j{};    // Переменная j инициализируется нулем
int* p;     // Переменная p имеет неопределенное значение
int* q{};   // Переменная q инициализируется константой nullptr
```

Однако *сужающие* инициализации, т.е. уменьшающие точность или модифицирующие передаваемое значение, в фигурных скобках запрещены. Например:

```
int x1(5.3);    // ОК, но x1 становится равным 5
int x2 = 5.3;   // ОК, но x2 становится равным 5
int x3{5.0};    // ОШИБКА: сужение
int x4 = {5.3}; // ОШИБКА: сужение
char c1{7};     // ОК: несмотря на то что 7 — целое значение,
                // это не сужение
char c2{999999}; // ОШИБКА: сужение (если 99999 выходит
                // за диапазон char)
std::vector<int> v1 { 1, 2, 4, 5 };    // ОК
std::vector<int> v2 { 1, 2.3, 4, 5.6 }; // ОШИБКА: сужение double в int
```

Легко видеть, что для проверки сужения могут рассматриваться даже текущие значения, доступные на этапе компиляции. Как написал Бьярне Страуструп (Bjarne Stroustrup) в документе [*Stroustrup.FAQ*] об этом примере: “Способ, которым стандарт C++11 позволяет избежать множества несогласованностей из-за сужающих преобразований, основан на анализе, когда это возможно, реальных значений (как число 7 в предыдущем примере), а не только типов этих значений. Если значение представляет целевой тип точно, то преобразование не является сужающим. Обратите внимание на то, что преобразование числа с плавающей точкой в целое всегда считается сужающим — даже преобразование 7.0 а 7”.

Для поддержки концепции списков инициализации для пользовательских типов стандарт C++11 предусматривает класс `std::initializer_list<>`. Его можно использовать для инициализации с помощью списка значений или в любом другом месте, где требуется список значений. Например:

```
void print (std::initializer_list<int> vals)
{
    for (auto p=vals.begin(); p!=vals.end(); ++p)
    {
        //обработка списка значений
        std::cout << *p << "\n";
    }
}

print ({12,3,5,7,11,13,17}); // передача списка значений функции print()
```

При наличии конструкторов как с конкретным количеством аргументов, так и со списком инициализации предпочтение отдается версии со списком инициализации.

```
class P
{
public:
    P(int,int);
    P(std::initializer_list<int>);
};

P p(77,5);    // вызов P::P(int,int)
P q{77,5};    // вызов P::P(initializer_list)
P r{77,5,42}; // вызов P::P(initializer_list)
P s = {77,5}; // вызов P::P(initializer_list)
```

Если бы конструктора со списком инициализации не было, то для инициализации объектов `q` и `s` был бы вызван конструктор, получающий два аргумента типа `int`, а инициализация объекта `r` была бы некорректной.

Благодаря спискам инициализации теперь ключевое слово `explicit` относится и к конструкторам, принимающим больше одного аргумента. Следовательно, теперь можно запретить автоматические преобразования типов нескольких значений, которые использовались также при инициализации с помощью синтаксиса присваивания.

```
class P
{
public:
    P(int a, int b) {
        ...
    }
}
```

```

    explicit P(int a, int b, int c) {
        ...
    }
};

P x(77,5);           // OK
P y(77,5);           // OK
P z {77,5,42};       // OK
P v = {77,5};        // OK (неявное преобразование типа допускается)
P w = {77,5,42};     // ОШИБКА из-за ключевого слова explicit
                    // (неявное преобразование типа не допускается)

void fp(const P&);

fp({47,11});         // OK, неявное преобразование {47,11} в P
fp({47,11,3});       // ОШИБКА из-за ключевого слова explicit
fp(P{47,11});        // OK, явное преобразование {47,11} в P
fp(P{47,11,3});      // OK, явное преобразование {47,11,3} в P

```

Аналогично в конструкторах с ключевым словом `explicit`, получающих список инициализации, запрещены неявные преобразования списков инициализации — как пустых, так и содержащих одно или несколько значений.

3.1.4. Диапазонные циклы `for`

Стандарт C++11 ввел новую форму цикла `for`, который перебирает все элементы в заданном диапазоне, массиве или коллекции. В других языках программирования такая форма цикла называется *foreach*. Общая синтаксическая конструкция этого цикла имеет следующий вид:

```

for ( decl : coll ) {
    операторы
}

```

где *decl* — объявление каждого элемента перебираемой коллекции *coll*, и к каждому элементу применяются указанные операторы. Например, следующий цикл применяет к каждому значению передаваемого списка инициализации оператор, выводящий это значение в стандартный поток вывода `cout`:

```

for ( int i : { 2, 3, 5, 7, 9, 13, 17, 19 } ) {
    std::cout << i << std::endl;
}

```

Для умножения каждого элемента `elem` вектора `vec` на 3 можно написать следующий код:

```

std::vector<double> vec;
...
for ( auto& elem : vec ) {
    elem *= 3;
}

```

Здесь важно подчеркнуть, что переменная `elem` объявлена как ссылка, потому что в противном случае операторы в теле цикла `for` применяются к локальным копиям элементов в векторе (что иногда бывает полезным).

Это значит, что, для того чтобы избежать вызова копирующего конструктора и деструктора для каждого элемента, необходимо объявить текущий элемент как константную ссылку. Таким образом, обобщенная функция для вывода всех элементов коллекции может быть реализована следующим образом:

```
template <typename T>
void printElements (const T& coll)
{
    for (const auto& elem : coll) {
        std::cout << elem << std::endl;
    }
}
```

Здесь диапазонный цикл `for` эквивалентен следующей конструкции:

```
{
    for (auto _pos=coll.begin(); _pos != coll.end(); ++_pos ) {
        const auto& elem = *_pos;
        std::cout << elem << std::endl;
    }
}
```

В общем случае диапазонный цикл `for`, объявленный как

```
for ( decl : coll ) {
    операторы
}
```

эквивалентен следующей конструкции (если объект `coll` имеет члены `begin()` и `end()`):

```
{
    for (auto _pos=coll.begin(), _end=coll.end(); _pos!=_end; ++_pos ) {
        decl = *_pos;
        операторы
    }
}
```

или, если таких членов нет, следующей конструкции, в которой используются глобальные функции `begin()` и `end()`, принимающие объект `coll` как аргумент:

```
{
    for (auto _pos=begin(coll), _end=end(coll); _pos!=_end; ++_pos ) {
        decl = *_pos;
        операторы
    }
}
```

В результате появляется возможность использовать диапазонный цикл `for` даже в случае списка инициализации, поскольку шаблонный класс `std::initializer_list<>` содержит члены `begin()` и `end()`.

Кроме того, существует правило, позволяющее использовать массивы в стиле языка C, имеющие известный размер. Например, программа

```
int array[] = { 1, 2, 3, 4, 5 };

long sum=0; // накапливает сумму всех элементов
for (int x : array) {
    sum += x;
}

for (auto elem : { sum, sum*2, sum*4 }) { // выводим на экран
                                         // несколько произведений суммы
                                         // на число
    std::cout << elem << std::endl;
}
```

выводит следующие результаты:

```
15
30
60
```

Обратите внимание на то, что при инициализации элементов с помощью конструкции *decl* в цикле *for* явное преобразование типа становится невозможным. Таким образом, следующий код скомпилирован не будет:

```
class C
{
public:
    explicit C(const std::string& s); // явное(!) преобразование типа
                                     // из класса string
    ...
};

std::vector<std::string> vs;
for (const C& elem : vs) { // ОШИБКА, преобразование класса string
                          // в класс C не определено
    std::cout << elem << std::endl;
}
```

3.1.5. Семантика перемещения и rvalue-ссылки

Одним из наиболее важных новшеств в стандарте C++11 является поддержка семантики перемещения. Это средство позволяет еще больше приблизиться к основной цели языка C++ и предотвратить создание ненужных копий и временных объектов.

Новое средство является настолько сложным, что, пытаясь дать краткое описание предмета, я все же рекомендую прочитать более подробное введение в эту тему³.

Рассмотрим следующий пример:

³ Это введение основано на работах [Abrahams:RValues] (с любезного разрешения Дэйва Абрамса), [Becker:RValues] (с любезного разрешения Томаса Беккера) и нескольких электронных письмах из переписки с Даниэлем Крюглером, Дитмаром Кюлем и Йенсом Маурером.


```
...
coll.insert(std::move(x)); // перемещает (или копирует) содержимое x
                           // в объект coll
```

С помощью функции `std::move()`, объявленной в заголовке `<utility>`, объект `x` можно *переместить*, а не копировать. Однако сама функция `std::move()` не выполняет перемещение, она просто преобразовывает свой аргумент в так называемую *rvalue-ссылку* (*rvalue reference*), представляющую собой тип, объявленный с двумя амперсандами: `X&&`. Этот новый тип представляет *rvalue-значения* (анонимные временные объекты, которые могут появляться только в правой части оператора присваивания), допускающие модификацию. По соглашению объект `x` в этом случае считается временным объектом, который уже не нужен, поэтому его содержимое и/или ресурсы можно захватить.

Теперь коллекция может использовать перегруженные версии функции `insert()`, работающей с *rvalue-ссылками*.

```
namespace std {
    template <typename T, ...> class multiset {
    public:
        ... insert (const T& x); // для lvalue-значений: копирует значение
        ... insert (T&& x);      // for rvalue-значений: перемещает значение
        ...
    };
}
```

Версия для *rvalue-значений* теперь может быть оптимизирована, чтобы ее реализация *захватывала* (*steals*) содержимое объекта `x`. Однако для этого нужна помощь со стороны типа объекта `x`, поскольку только члены объекта `x` имеют доступ к его внутренним компонентам. Так, например, для инициализации вставленного элемента можно использовать внутренние массивы и указатели из объекта `x`. Если класс объекта `x` является сложным, это может значительно повысить быстродействие программы. В противном случае нам пришлось бы копировать элемент за элементом. Для инициализации нового внутреннего элемента мы просто вызываем так называемый перемещающий конструктор из класса `X`, который захватывает значение переданного аргумента для инициализации нового объекта. Все сложные типы должны — как это сделано в стандартной библиотеке C++ — иметь такой специальный конструктор, осуществляющий перемещение содержимого существующего элемента в новый элемент.

```
class X {
    public:
        X (const X& lvalue); // копирующий конструктор
        X (X&& rvalue);      // перемещающий конструктор
        ...
};
```

Например, перемещающий конструктор для строк обычно просто присваивает существующий внутренний массив символов новому объекту, а не создает новый массив и не копирует все элементы. То же самое относится и ко всем классам коллекций: вместо создания копий всех элементов мы просто присваиваем внутреннюю память новому объекту. Если в классе нет перемещающего конструктора, используется копирующий конструктор.

Кроме того, следует сделать так, чтобы любая модификация — особенно уничтожение — передаваемого объекта, содержимое которого было *захвачено*, не оказывало

влияния на состояние нового объекта, который стал владельцем значения. Для этого обычно достаточно стереть содержимое передаваемого аргумента (например, присвоив константу `nullptr` его внутреннему члену, ссылающемуся на его элементы).

Стирание содержимого объекта, к которому применяется семантика перемещения, вообще говоря, не является необходимым, но если этого не сделать, то весь механизм становится практически бесполезным. Все классы стандартной библиотеки C++ гарантируют, что после перемещения объект пребывает в *корректном, но неопределенном состоянии*. Иначе говоря, вы можете впоследствии присваивать ему новые значения, но его текущее состояние не определено. Контейнеры из библиотеки STL гарантируют, что после перемещения значений они остаются пустыми.

Точно так же любой нетривиальный класс должен содержать оператор копирующего присваивания и оператор перемещающего присваивания.

```
class X {
public:
    X& operator= (const X& lvalue); // оператор копирующего присваивания
    X& operator= (X&& rvalue);     // оператор перемещающего присваивания
    ...
};
```

Для строк и коллекций эти операторы можно было бы реализовать так, чтобы они просто меняли местами внутреннее содержимое и ресурсы. Однако в этом случае также следует стереть содержимое объекта `*this`, потому что он может владеть ресурсами, например блоками, которые необходимо как можно быстрее освободить. Как и в предыдущих случаях, с формальной точки зрения семантика перемещения не требует этого, но, например, контейнерные классы в стандартной библиотеке C++ делают это в обязательном порядке.

В заключение сделаем два замечания о новых средствах языка: 1) правила перегрузки для `rvalue`- и `lvalue`-ссылок и 2) возврат `rvalue`-ссылок.

Правила перегрузки для `rvalue`- и `lvalue`-ссылок

Правила перегрузки для `rvalue`- и `lvalue`-значений формулируются следующим образом⁴.

- Если реализуется только функция

```
void foo(X&);
```

без `foo(X&&)`, то ее поведение следует стандарту C++98: функцию `foo()` можно вызывать для `lvalue`-значений, но не для `rvalue`-значений.

- Если реализуется функция

```
void foo(const X&);
```

без функции `void foo(X&&)`, то ее поведение следует стандарту C++98: функцию `foo()` можно вызывать как для `rvalue`-значений, так и для `lvalue`-значений.

- Если реализуются функции

```
void foo(X&);
void foo(X&&);
```

⁴ Данная формулировка принадлежит Томасу Беккеру.

или

```
void foo(const X&);  
void foo(X&&);
```

то rvalue- и lvalue-значения используются по-разному. Версия функции для rvalue-значений может и обязана использовать семантику перемещения. Таким образом, она может *захватывать* внутреннее состояние и ресурсы передаваемого аргумента.

- Если реализуется функция

```
void foo(X&&);
```

но ни `void foo(X&)`, ни `void foo(const X&)` не реализованы, то функция `foo()` может вызываться для rvalue-значений, но попытка вызвать ее для lvalue-значения вызовет ошибку компиляции. Таким образом, здесь предусматривается только семантика перемещения. Эта возможность используется в библиотеке: например, уникальными указателями (см. раздел 5.2.5), файловыми потоками (см. раздел 15.9.2) или строковыми потоками (см. раздел 15.10.2).

Это означает, что если класс не предусматривает семантику перемещения и содержит только обычный копирующий конструктор и оператор копирующего присваивания, то он может использоваться только для rvalue-ссылок. Таким образом, функция `std::move()` реализует семантику перемещения, если она предусмотрена в классе, или семантику копирования в противном случае.

Возврат rvalue-ссылок

Функцию `move()` не обязательно применять к возвращаемым значениям. В соответствии с правилами языка в стандарте определено, что для кода ⁵

```
X foo ()  
{  
  X x;  
  ...  
  return x;  
}
```

гарантируется следующее поведение:

- Если класс `X` имеет доступный копирующий конструктор или конструктор перемещения, компилятор может игнорировать копию. Это так называемая (*именованная*) *оптимизация возвращаемого значения* ((named) return value optimization — (N) RVO). Она была определена еще до появления стандарта C++11 и поддерживалась большинством компиляторов.
- В противном случае, если в классе `X` есть перемещающий конструктор, объект `x` перемещается.

⁵ Эта формулировка принадлежит Дэйву Абрамсу.

- В противном случае, если в классе `X` есть копирующий конструктор, объект `x` копируется.
- В противном случае возникает ошибка компиляции.

Отметим, что возврат `rvalue`-ссылки является ошибкой, если возвращаемый объект является локальным и нестатическим.

```
X&& foo ()
{
    X x;
    ...
    return x; // ОШИБКА: возвращает ссылку на несуществующий объект
}
```

`Rvalue`-ссылка — это разновидность ссылки, и ее возвращение в момент, когда она ссылается на локальный объект, означает, что вы возвращаете ссылку на объект, которого больше нет. Применение функции `std::move()` на это никак не влияет.

3.1.6. Новые строковые литералы

Начиная с версии C++11 можно определять литералы как для неформатированных строк, так и для многобайтовых строк или строк, состоящих из широких символов.

Литералы для неформатированных строк

Такие неформатированные строки позволяют определить последовательность символов, просто записывая ее содержимое в виде набора символов. Это позволяет избавиться от множества управляющих символов, используемых для маскировки специальных символов.

Неформатированная строка начинается с символов `R" (` и заканчивается символами `) "`. Такая строка может включать переход на новую строку. Например, обычный строковый литерал, содержащий две обратных косых черты и символ `n`, можно определить как обычный строковый литерал

```
"\\\\\\n"
```

или как неформатированный строковый литерал

```
R" (\\n) "
```

Для вставки символов `) "` в неформатированную строку можно использовать разделитель. Таким образом, полный синтаксис неформатированной строки выглядит как `R"delim ( . . . ) delim"`, где *delim* — последовательность, состоящая из не более чем 16 основных символов, исключая обратную косую черту, пробелы и скобки.

Например, неформатированный строковый литерал

```
R"nc(a\
    b\nc() "
    )nc";
```

эквивалентен следующему обычному строковому литералу:

```
"a\\n      b\\nc()\\n      "
```

Следовательно, эта строка содержит символ `a`, обратную косую черту, символ перехода на новую строку, несколько пробелов, символ `b`, обратную косую черту, символ `n`, символ `c`, символ двойной кавычки, символ перехода на новую строку и несколько пробелов.

Неформатированные строковые литералы особенно полезны при определении регулярных выражений. (Детали изложены в главе 14.)

Закодированные строковые литералы

С помощью *префикса кодировки* (encoding prefix) можно указать конкретное кодирование символов в строковых литералах. В стандарте определены следующие префиксы кодировки.

- Префикс `u8` определяет кодировку UTF-8. Строковый литерал в кодировке UTF-8 инициализируется заданными символами в соответствии с кодировкой UTF-8. Эти символы должны иметь тип `const char`.
- Префикс `u` определяет строковый литерал, состоящий из символов, имеющих тип `char16_t`.
- Префикс `U` определяет строковый литерал, состоящий из символов, имеющих тип `char32_t`.
- Префикс `L` определяет строковый литерал, состоящий из символов, имеющих тип `wchar_t`.

Например::

```
L"hello" // определяет строку "hello" как строковый литерал типа wchar_t
```

Перед начальным символом `R` в неформатированной строке может стоять префикс кодировки.

Детали использования разных кодировок для интернационализации описаны в главе 16.

3.1.7. Ключевое слово `noexcept`

Стандарт C++11 предусматривает ключевое слово `noexcept`. Его можно использовать для того, чтобы указать, что функция не может генерировать исключения или не предназначена для этого. Например, код

```
void foo () noexcept;
```

объявляет, что функция `foo()` не может генерировать исключения. Если исключение не было обработано локально в функции `foo()`, т.е. если функция `foo()` все же сгенерировала исключение, программа прекращает работу, вызывая функцию `std::terminate()`, которая по умолчанию вызывает функцию `std::abort()` (см. раздел 5.8.2).

Ключевое слово `noexcept` решает множество проблем, связанных со спецификациями исключений. Пр процитируем работу [N3051:DeprExcSpec] (с любезного разрешения Дуга Грегора).

- Проверка во время выполнения программы. Спецификации исключения в языке C++ проверяются во время выполнения программы, а не на этапе компиляции, поэтому у программиста нет гарантии, что все исключения будут обработаны. Сбой во время выполнения (приводящий к вызову функции `std::unexpected()`) сам по себе не подлежит восстановлению.
- Излишние затраты ресурсов при выполнении программы. Проверка на этапе выполнения программы требует, чтобы компилятор сгенерировал дополнительный код, который мешает оптимизации.
- Недопустимость в обобщенном коде. В обобщенном коде обычно невозможно знать заранее, какие типы исключений могут генерироваться операторами, принадлежащими шаблонным аргументам, поэтому невозможно написать точную спецификацию исключения.

На практике оказались полезными только две спецификации, связанные с генерацией исключений: оператор может генерировать любое исключение или не должен генерировать исключения вовсе. Первое требование выражается в игнорировании спецификации исключений вообще, а второе можно выразить с помощью оператора `throw()`, хотя это редко делают из-за снижения быстродействия программы.

Ключевое слово `noexcept` не требует разворачивания стека, поэтому программисты теперь могут явно гарантировать отсутствие исключений без дополнительных затрат. В результате использование спецификаций исключений из стандарта C++11 исключено.

Можно даже указать условие, при котором функция не генерирует никаких исключений. Например, для любого типа `Type` глобальная функция `swap()` обычно определяется следующим образом:

```
void swap (Type& x, Type& y) noexcept(noexcept(x.swap(y)))
{
    x.swap(y);
}
```

Здесь в спецификаторе `noexcept(...)` можно задать булево выражение, при котором функция не может генерировать исключение. Если в спецификаторе `noexcept` не указано никаких условий, подразумевается условие `noexcept(true)`.

В данном случае условие имеет вид `noexcept(x.swap(y))`. Здесь используется *оператор* `noexcept`, который возвращает значение `true`, если вычисляемое выражение, заданное в скобках, не может генерировать исключения. Таким образом, глобальная функция `swap()` указывает, что она не генерирует никаких исключений, если функция-член `swap()`, вызванная для первого аргумента, не генерирует исключений.

Другой пример связан с оператором перемещающего присваивания, который применяется к паре значений следующим образом:

```
pair& operator= (pair&& p)
    noexcept(is_nothrow_move_assignable<T1>::value &&
             is_nothrow_move_assignable<T2>::value);
```

Здесь используется свойство типа `is_nothrow_move_assignable`, которое проверяет, можно ли применять к передаваемому типу оператор перемещающего присваивания, который не может генерировать исключения (см. раздел 5.4.2).

В соответствии с документом [N3279:LibNoexcept], ключевое слово `noexcept` включено в библиотеку для реализации консервативного подхода (слова и фразы, *набранные курсивом*, цитируются буквально).

- *Каждая библиотечная функция... которая ...не может генерировать исключения и не задает никакого непредсказуемого поведения — например, вследствие нарушения предусловия — должна быть помечена безусловной спецификацией `noexcept`.*
- *Если можно доказать, что библиотечная функция обмена, перемещающий конструктор или оператор перемещающего присваивания... не могут генерировать исключение путем применения оператора `noexcept`, они должны быть помечены условной спецификацией `noexcept`. Ни одна другая функция не должна использовать условную спецификацию `noexcept`.*
- *Ни один библиотечный деструктор не должен генерировать исключения. Он должен использовать неявно указанную спецификацию, гарантирующую отсутствие генерации исключений.*
- *Библиотечные функции, разработанные с учетом совместимости с языком C... могут быть помечены безусловной спецификацией `noexcept`.*

Отметим, что спецификация `noexcept` преднамеренно не применяется к функциям языка C++, имеющим предусловие, нарушение которого может привести к непредсказуемому поведению. Это позволяет обеспечить “безопасный режим” реализации библиотек путем генерирования исключения “нарушение предусловия” в случае неправильного использования функции.

На протяжении книги я обычно пропускаю спецификации `noexcept`, чтобы не загромождать изложение.

3.1.8. Ключевое слово `constexpr`

Начиная с версии C++11 с помощью ключевого слова `constexpr` можно отмечать выражения, вычисляемые на этапе компиляции. Например:

```
constexpr int square (int x)
{
    return x * x;
}
float a[square(9)]; // OK в версии C++11:
                  // массив a содержит 81 элемент
```

Это ключевое слово решает проблему, возникшую в стандарте C++98 при использовании предельных числовых значений (см. раздел 5.3). До стандарта C++11 такое выражение, как

```
std::numeric_limits<short>::max()
```

не могло использоваться как целочисленная константа, хотя и было функционально эквивалентным макросу `INT_MAX`. С появлением стандарта C++11 такие выражения объявляются с ключевым словом `constexpr`. Например, его можно использовать для объявления массива и вычислений на этапе компиляции (метапрограммирования):

```
std::array<float, std::numeric_limits<short>::max()> a;
```

На протяжении книги я обычно пропускаю спецификации `constexpr`, чтобы не загромождать изложение.

3.1.9. Новые возможности шаблонов

Вариативные шаблоны

В стандарте C++11 шаблоны могут иметь параметры, принимающие переменное количество шаблонных аргументов. Эта возможность называется *вариативными шаблонами* (variadic templates). В следующем примере их можно использовать для вызова функции `print()` с аргументами разных типов.

```
void print ()
{
}

template <typename T, typename... Types>
void print (const T& firstArg, const Types&... args)
{
    std::cout << firstArg << std::endl; // выводим на экран первый аргумент
    print(args...);                     // вызываем print() для
// остальных аргументов
}
```

При передаче одного или нескольких аргументов используется шаблонная функция, в которой отдельное определение первого аргумента позволяет выводить его на экран, а затем рекурсивно вызывать функцию `print()` с оставшимися аргументами. Для завершения рекурсии предоставляется нешаблонная перегруженная версия функции `print()`.

Например, вызов функции

```
print (7.5, "hello", std::bitset<16>(377), 42);
```

приводит к следующему результату (см. раздел 12.5.1, в котором подробно описаны битовые множества):

```
7.5
hello
00000000101111001
42
```

Отметим, что корректность этого примера в настоящее время также является предметом обсуждения. Причина заключается в том, что вариативная форма с одним аргументом формально совпадает с невариативной формой с одним аргументом; однако компиляторы обычно допускают следующий код:

```
template <typename T>
void print (const T& arg)
{
    std::cout << arg << std::endl;
}
```

```
template <typename T, typename... Types>
void print (const T& firstArg, const Types&... args)
{
    std::cout << firstArg << std::endl; // выводим на экран первый аргумент
    print(args...);                     // вызываем print() для
// остальных аргументов
}
```

В вариативных шаблонах оператор `sizeof...(args)` возвращает количество аргументов.

Эту функциональную возможность интенсивно использует класс `std::tuple<>` (см. раздел 5.1.2).

Шаблонный псевдоним (шаблонный оператор `typedef`)

В стандарте C++11 поддерживаются определения шаблонных (частичных) типов. Однако, поскольку все подходы, основанные на использовании ключевого слова `typename`, по некоторой причине оказались неработоспособными, было введено ключевое слово `using` и термин *псевдоним шаблона* (alias template). Например, после операторов

```
template <typename T>

using Vec = std::vector<T, MyAlloc<T>>; // стандартный вектор,
// использующий собственный механизм
// распределения памяти
```

выражение

```
Vec<int> coll;
```

становится эквивалентным выражению

```
std::vector<int, MyAlloc<int>> coll;
```

Другой пример можно найти в разделе 5.2.5.

Другие новые шаблонные средства

В стандарте C++11 шаблонные функции (см. раздел 3.2) могут иметь шаблонные аргументы, заданные по умолчанию. Кроме того, локальные типы теперь можно использовать как шаблонные аргументы, а функции с внутренним связыванием теперь можно использовать в качестве аргументов для нетипизированных шаблонов указателей или ссылок на функции.

3.1.10. Лямбда-выражения и лямбда-функции

В стандарте C++11 появились *лямбда-выражения* и *лямбда-функции*, позволяющие создавать определения подставляемых функций, которые можно использовать в качестве параметра или локального объекта.

Лямбда-выражения и лямбда-функции изменяют способ использования стандартной библиотеки C++. Например, в разделах 6.9 и 10.3 показано, как использовать лямбда-выражения и лямбда-функции с алгоритмами и контейнерами из библиотеки STL. В разделе 18.1.2 продемонстрировано, как с помощью лямбда-выражений и лямбда-функций определить код, который может работать в параллельном режиме.

Синтаксис лямбда-выражений и лямбда-функций

Лямбда-функция — это описание функциональной возможности, которую можно определить в операторе и выражении. Таким образом, лямбда-функцию можно использовать в качестве подставляемой.

Минимальная лямбда-функция не имеет параметров и просто делает что-то. Например:

```
[ ] {
    std::cout << "hello lambda" << std::endl;
}
```

Эту функцию можно вызывать непосредственно

```
[ ] {
    std::cout << "hello lambda" << std::endl;
} (); // выводит на экран "hello lambda"
```

или передавать вызываемым объектам

```
auto l = [ ] {
    std::cout << "hello lambda" << std::endl;
};
...
l(); // выводим на экран "hello lambda"
```

Как видим, лямбда-функции всегда предшествует так называемый *инициатор лямбда-функции* (lambda introducer): квадратные скобки, внутри которых можно определить так называемый захват для доступа к нестатическим внешним объектам в лямбда-функции. Если доступ к внешним данным не нужен, квадратные скобки остаются пустыми, как в данном случае. В лямбда-функциях можно использовать статические объекты, например `std::cout`.

Между инициатором лямбда-функции и телом лямбда-функции можно указать параметры, ключевое слово `mutable`, спецификацию исключения, спецификаторы атрибутов и тип возвращаемого значения. Все это не обязательно, но если один из перечисленных пунктов присутствует, круглые скобки для параметров становятся обязательными. Таким образом, синтаксис лямбда-функции выглядит либо так:

```
[...] {...}
```

либо так:

```
[...] (...) mutableopt throwSpecopt ->retTypeopt {...}
```

Лямбда-функция может иметь параметры, указанные в скобках, как обычная функция.

```
auto l = [] (const std::string& s) {  
    std::cout << s << std::endl;  
};  
l("hello lambda"); // выводит на экран "hello lambda"
```

Однако следует подчеркнуть, что лямбда-функции не могут быть шаблонными. В них всегда необходимо указывать все типы.

Лямбда-функция может возвращать какой-нибудь объект. Если тип возвращаемого объекта не указан, он выводится из его значения. Например, следующая функция возвращает значение типа `int`:

```
[] {  
    return 42;  
}
```

Для указания типа возвращаемого значения можно использовать новую синтаксическую конструкцию языка C++, которая применяется и для обычных функций (см. раздел 3.1.12). Например, следующая лямбда-функция возвращает число 42.0:

```
[] () -> double {  
    return 42;  
}
```

В данном случае необходимо указать тип возвращаемого значения после обязательных в данном случае скобок, предназначенных для аргументов, и символов “->.”

Между параметрами и указанием типа возвращаемого значения или телом можно также задать спецификацию исключения, как при работе с обычной функцией. Однако для обычных функций спецификации исключения объявлены устаревшими (см. раздел 3.1.7).

Захваты (доступ к внешней области видимости)

В инициаторе лямбда-функции (квадратных скобках перед лямбда-функцией) можно задать *список захвата* (capture) для доступа к данным из внешней области видимости, которые не передаются как аргументы.

- Символы [=] означают, что внешняя область видимости передается в лямбда-функцию по значению. Таким образом, можно прочитать, но не модифицировать все данные, которые были доступными для чтения при определении лямбда-функции.
- Символы [&] означают, что внешняя область видимости передается в лямбда-функцию по ссылке. Таким образом, при определении лямбда-функции можно изменять данные, которые были корректными в момент определения лямбда-функции, при условии, что их можно изменять в принципе.

Кроме того, для каждого объекта в лямбда-функции необходимо указать режим доступа к нему: по значению или по ссылке. Это позволяет ограничивать доступ и смешивать разные режимы. Например, операторы

```
int x=0;  
int y=42;  
auto qq = [x, &y] {  
    std::cout << "x: " << x << std::endl;  
    std::cout << "y: " << y << std::endl;  
};
```

```
        ++y; // OK
    };
x = y = 77;
qqq();
qqq();
std::cout << "final y: " << y << std::endl;
```

приводят к следующим результатам:

```
x: 0
y: 77
x: 0
y: 78
final y: 79
```

Поскольку объект `x` передается по значению, его нельзя модифицировать в лямбда-функции; оператор `++x` в лямбда-функции не скомпилируется. Поскольку объект `y` передается по ссылке, его можно модифицировать, поэтому двойной вызов лямбда-функции дважды увеличит присвоенное изначально значение 77.

Вместо списка захвата `[x, &y]` можно написать список `[=, &y]` и передать объект `y` по ссылке, а все остальные объекты — по значению.

Для того чтобы смешать передачу по ссылке и по значению, лямбда-функцию можно объявить с ключевым словом `mutable`. В этом случае объекты передаются по значению, но в функции-объекте, определенной в лямбда-функции, переданное значение можно изменить. Например, код

```
int id = 0;
auto f = [id] () mutable {
    std::cout << "id: " << id << std::endl;
    ++id; // OK
};
id = 42;
f();
f();
f();
std::cout << id << std::endl;
```

выводит следующие результаты:

```
id: 0
id: 1
id: 2
42
```

Лямбда-функцию можно сравнить с функциональным объектом (см. раздел 6.10).

```
class {
private:
    int id; // копирование внешнего id
public:
    void operator() () {
        std::cout << "id: " << id << std::endl;
```

```

    ++id; // OK
}
};

```

Благодаря ключевому слову `mutable` оператор вызова функции `operator ()` определен как неконстантная функция-член, т.е. существует возможность изменять значение переменной `id`. Итак, благодаря ключевому слову `mutable` лямбда-функция сохраняет текущее состояние, даже если состояние передается по значению. Без ключевого слова `mutable`, что является обычной ситуацией, оператор вызова функции `operator ()` становится константной функцией-членом, и вы имеете право лишь читать объекты, переданные по значению. Другой пример использования ключевого слова `mutable` с лямбда-функциями приведен в разделе 10.3.2, в котором также освещены возможные проблемы.

Типы лямбда-функций

Типом лямбда-функции является анонимный функциональный объект (или функтор), являющийся уникальным для каждого лямбда-выражения. Таким образом, для объявления объектов такого типа необходимы шаблоны или ключевое слово `auto`. Если необходим тип, можно использовать ключевое слово `decltype` (см. раздел 3.1.11), которое, например, требуется при передаче лямбда-функции в качестве хеш-функции или критерия сортировки для ассоциативных или неупорядоченных контейнеров. Подробности описаны в разделах 6.9 и 7.9.7.

В качестве альтернативы для указания общего типа, предназначенного для функционального программирования, можно использовать шаблонный класс `std::function<>`, предоставляемый стандартной библиотекой языка C++ (см. раздел 5.4.4). Этот шаблонный класс обеспечивает единственный способ задать тип значения, возвращаемого лямбда-функцией:

```

// lang/lambda1.cpp

#include<functional>
#include<iostream>

std::function<int(int,int)> returnLambda ()
{
    return [] (int x, int y) {
        return x*y;
    };
}

int main()
{
    auto lf = returnLambda();
    std::cout << lf(6,7) << std::endl;
}

```

Ниже приведен результат работы этой программы.

3.1.11. Ключевое слово `decltype`

С помощью ключевого слова `decltype` можно позволить компилятору самому распознать тип выражения. Это реализация часто используемого оператора `typeof`. Поскольку существующие реализации оператора `typeof` противоречивы и неполны, стандарт C++11 ввел новое ключевое слово. Например:

```
std::map<std::string, float> coll;
decltype(coll)::value_type elem;
```

Одно из применений ключевого слова `decltype` — объявление типов возвращаемых значений (см. ниже). Другое применение относится к метапрограммированию (см. раздел 5.4.1) и передаче типа лямбда-функции (см. раздел 10.3.4).

3.1.12. Новый синтаксис объявления функций

Иногда тип значения, возвращаемого функцией, зависит от выражения, имеющего аргументы. Однако код наподобие

```
template <typename T1, typename T2>
decltype(x+y) add(T1 x, T2 y);
```

до появления стандарта C++11 был невозможен, поскольку тип возвращаемого значения в этом случае использует объекты, которые не были объявлены или которые еще не появились в области видимости.

В стандарте C++11 существует альтернативное объявление типа значения, возвращаемого функцией, которое размещается после списка параметров.

```
template <typename T1, typename T2>
auto add(T1 x, T2 y) -> decltype(x+y);
```

Это объявление использует тот же синтаксис, что и лямбда-функции при объявлении типов возвращаемых значений (см. раздел 3.1.10).

3.1.13. Перечисления с ограниченной областью видимости

Стандарт C++11 позволяет определять *перечисления с ограниченной областью видимости* (scoped enumerations), известные также как *строгие перечисления* (strong enumerations) или *классы перечислений* (enumeration classes). Эти перечисления являются более точной реализацией перечисляемых значений (перечислителей) в языке C++.

Например:

```
enum class Salutation : char { mr, ms, co, none };
```

Следует подчеркнуть, что ключевое слово `class` после `enum` является обязательным.

Перечисления с ограниченной областью видимости имеют следующие преимущества.

- Неявные преобразования в тип `int` и из типа `int` невозможны.
- Значения вроде `mr` не являются частью области видимости, в которой объявлено перечисление. Вместо него следует использовать конструкцию `Salutation::mr`.

- Можно явно определить базовый тип (в данном случае `char`) и гарантировать размер элементов перечисления (если пропустить выражение “: `char`”, то по умолчанию будет использоваться тип `int`).
- Возможны предваряющие объявления перечислимых типов, исключающие необходимость повторной компиляции модулей с новыми значениями перечислений при условии, что используется только данный тип.

Обратите внимание на то, что свойство типа `std::underlying_type` позволяет определить базовый тип перечисления (см. раздел 5.4.2).

Примерами *перечислений с ограниченной областью видимости* являются условные коды ошибок (см. раздел 4.3.2).

3.1.14. Новые фундаментальные типы данных

В стандарте C++11 определены новые фундаментальные типы данных.

- `char16_t` и `char32_t` (см. раздел 16.1.3)
- `long long` и `unsigned long long`
- `std::nullptr_t` (см. раздел Section 3.1.1)

3.2. Старые “новые” средства языка

Несмотря на то что стандарту C++98 уже больше десяти лет, программисты все еще иногда удивляются некоторым средствам языка. Рассмотрим некоторые из них.

Нетипизированные шаблонные параметры

Помимо типизированных параметров допускаются нетипизированные. Нетипизированный параметр рассматривается как часть типа. Например, стандартному классу `bitset<>` (см. раздел 12.5) можно передать количество битов в виде шаблонного аргумента. Следующие операторы определяют два битовых поля: одно из 32 битов, другое — из 50:

```
bitset<32> flags32; // битовое поле из 32 битов
bitset<50> flags50; // битовое поле из 50 битов
```

Эти битовые множества имеют разные типы, потому что они используют разные шаблонные аргументы. Таким образом, их нельзя присваивать друг другу и сравнивать между собой, не предусмотрев соответствующего преобразования типа.

Шаблонные параметры, заданные по умолчанию

Шаблонные классы могут иметь аргументы, заданные по умолчанию. Например, следующее объявление позволяет объявлять объекты класса `MyClass` с одним или двумя шаблонными аргументами:

```
template <typename T, typename container = vector<T>>
class MyClass;
```

Если передать только один аргумент, то вторым будет использоваться параметр, заданный по умолчанию.

```
MyClass<int> x1; // эквивалент MyClass<int, vector<int>>
```

Отметим, что шаблонные аргументы, заданные по умолчанию, можно определять с помощью предыдущих аргументов.

Ключевое слово `typename`

Ключевое слово `typename` было введено для того, чтобы указать, что следующий за ним идентификатор обозначает тип. Рассмотрим следующий пример:

```
template <typename T>
class MyClass {
    typename T::SubType * ptr;
    ...
};
```

где ключевое слово `typename` используется для того, чтобы разъяснить, что `SubType` — это тип, определенный в классе `T`. Таким образом, `ptr` — это указатель на объект типа `T::SubType`. Без ключевого слова `typename` имя `SubType` рассматривалось бы как имя статического члена, а значит, выражение

```
T::SubType * ptr
```

означало бы умножение значения объекта `SubType` типа `T` на переменную `ptr`.

Поскольку в программе указано, что `SubType` — это тип, то любой тип, используемый вместо параметра `T`, должен иметь внутренний тип `SubType`. Например, использование типа `Q` в качестве шаблонного аргумента возможно только в том случае, если он содержит определение внутреннего типа `SubType`.

```
class Q {
    typedef int SubType;
    ...
};
```

```
MyClass<Q> x; // OK
```

В таком случае переменная `ptr`, являющаяся членом класса `MyClass<Q>`, представляла бы собой указатель на тип `int`. Однако подтип также может быть абстрактным типом данных, таким, как класс:

```
class Q {
    class SubType;
    ...
};
```

Отметим, что ключевое слово `typename` необходимо использовать всегда, если требуется квалифицировать идентификатор шаблона в качестве типа, даже если другая

интерпретация лишена смысла. Итак, общее правило в языке C++ утверждает, что любой идентификатор шаблона рассматривается как значение, если он не квалифицирован ключевым словом `typename`.

Помимо всего прочего, ключевое слово `typename` можно использовать вместо ключевого слова `class` в объявлении шаблона.

```
template <typename T> class MyClass;
```

Шаблонные члены

Функции-члены класса могут быть шаблонными. Однако шаблонные функции-члены не могут быть виртуальными. Например:

```
class MyClass {
    ...
    template <typename T>
    void f(T);
};
```

где выражение `MyClass::f` объявляет набор функций-членов с параметрами любого типа. Вы можете передавать ей любой аргумент, при условии, что его тип предусматривает все операторы, выполняемые в функции `f()`.

Это свойство часто используется для поддержки автоматического преобразования типов членов в шаблонных классах. Например, в следующем определении аргумент `x` функции `assign()` должен иметь точно такой же тип, как и вызывающий объект:

```
template <typename T>
class MyClass {
private:
    T value;
public:
    void assign (const MyClass<T>& x) { // x должен иметь такой же
                                      // тип, что и *this
        value = x.value;
    }
    ...
};
```

При таком объявлении было бы ошибкой использовать разные шаблонные типы для объектов в функции `assign()`, даже если предусмотрено автоматическое преобразование одного типа в другой.

```
void f()
{
    MyClass<double> d;
    MyClass<int> i;

    d.assign(d); // ОК
    d.assign(i); // ОШИБКА: i имеет тип MyClass<int>
                  // а требуется тип MyClass<double>
}
```

Задав другой шаблонный тип для функции-члена, можно ослабить требование строгого совпадения типов. Шаблонный аргумент функции-члена может иметь любой шаблонный тип. Тогда, поскольку эти типы допускают присваивание, можно написать следующий код:

```
template <typename T>
class MyClass {
private:
    T value;
public:
    template <typename X>                // Шаблонный член
    void assign (const MyClass<X>& x) { // Разрешены другие шаблонные типы
        value = x.getValue();
    }
    T getValue () const {
        return value;
    }
    ...
};

void f()
{
    MyClass<double> d;
    MyClass<int> i;

    d.assign(d); // OK
    d.assign(i); // OK (int можно присваивать переменной типа double)
}
```

Отметим, что аргумент *x* функции `assign()` теперь отличается от типа объекта `*this`. Таким образом, у нас нет непосредственного доступа к закрытым и защищенным членам класса `MyClass<>`. Вместо этого мы должны использовать что-то вроде функции `getValue()`.

Особым видом шаблонного члена является *шаблонный конструктор* (`template constructor`). Шаблонные конструкторы обычно используются для осуществления неявных преобразований типа при копировании объектов. Обратите внимание на то, что шаблонный конструктор не подавляет неявное объявление копирующего конструктора. Если типы совпадают точно, генерируется и вызывается неявный копирующий конструктор. Например:

```
template <typename T>
class MyClass {
public:
    // копирующий конструктор с неявным преобразованием типов
    // не подавляет автоматический копирующий конструктор
    template <typename U>
    MyClass (const MyClass<U>& x);
    ...
};

void f()
{
    MyClass<double> xd;
```

```

...
MyClass<double> xd2(xd); // вызывает неявно сгенерированный
                        // копирующий конструктор
MyClass<int> xi(xd);     // вызывает шаблонный конструктор
...
}

```

Здесь тип объекта `xd2` совпадает с типом объекта `xd` и поэтому инициализируется с помощью неявно сгенерированного копирующего конструктора. Тип объекта `xi` отличается от типа объекта `xd` и поэтому инициализируется шаблонным конструктором. Таким образом, если вы реализуете шаблонный конструктор, не забудьте предусмотреть конструктор по умолчанию, если его поведение по умолчанию вас не устраивает. Другой пример шаблонных членов класса приведен в разделе 5.1.1.

Вложенные шаблонные классы

Вложенные классы также могут быть шаблонными.

```

template <typename T>
class MyClass {
    ...
    template <typename T2>
    class NestedClass;
    ...
};

```

3.2.1. Неявная инициализация фундаментальных типов

Если вы используете синтаксис явного вызова конструктора без аргументов, объекты фундаментальных типов инициализируются нулем.

```

int i1;           // неопределенное значение
int i2 = int();   // инициализируется нулем
int i3{};         // инициализируется нулем (по стандарту C++11)

```

Это средство позволяет писать шаблонный код, гарантирующий, что переменные любого типа всегда будут иметь определенное значение, заданное по умолчанию. Например, в следующей функции инициализация гарантирует, что объект `x` фундаментального типа инициализируется нулем:

```

template <typename T>
void f()
{
    T x = T();
    ...
}

```

Если шаблонная функция выполняет принудительную инициализацию нулем, то значение, возвращаемое ею, называется *значением, инициализированным нулем* (zero initialized). В противном случае оно *инициализируется значением по умолчанию*.

3.2.2. Определение функции `main()`

Следует разъяснить один важный аспект языка, который часто понимают неправильно: корректные и переносимые версии функции `main()`. В соответствии со стандартом языка C++ переносимыми являются только два определения функции `main()`.

```
int main()
{
    ...
}
```

и

```
int main (int argc, char* argv[])
{
    ...
}
```

Здесь аргумент `argv` (массив аргументов командной строки) можно также определить как `char**`. Отметим, что в функции `main` в обязательном порядке требуется тип возвращаемого значения `int`.

Функцию `main()` не обязательно завершать оператором `return`. В отличие от языка C, в языке C++ в конце функции `main()` неявно определен оператор

```
return 0;
```

Это значит, что каждая программа, осуществляющая выход из функции `main()` без выполнения оператора `return`, считается завершенной успешно. Любое значение, отличающееся от нуля, означает ошибку (предопределенные коды ошибок перечислены в разделе 5.8.2). В связи с этим мои примеры в книге никогда не содержат оператор `return` в конце функции `main()`.

Для завершения программы на языке C++ без выхода из функции `main()` обычно следует использовать функции `exit()`, `quick_exit()` (начиная со стандарта C++11) или `terminate()`. Подробности описаны в разделе 5.8.2.